Library Management System

Context

A university library maintains books and other learning resources used by students for coursework, technical preparation, research, placements and general learning. Multiple physical copies of a book may be available, while some titles may have limited copies and high demand.

Currently, book requests, reservations, issue records and returns are coordinated through library registers, spreadsheets and informal communication. As usage grows, the university faces conflicting requests for limited copies, books being requested without considering availability, limited visibility into request and loan status, delays in reviewing and confirming requests, copies remaining unavailable after cancelled reservations or completed returns, temporary unavailability due to damage or repair, and different stakeholders requiring different information and capabilities.

Design and build a working prototype of a Library Management System. Your team must study the business situation, identify the relevant stakeholders, and determine the workflows, business rules and validations required for the system to operate reliably. The solution should support a meaningful library lending lifecycle, from an initial borrowing request to an appropriate outcome.

Multiple valid approaches are possible. Your solution should be internally consistent, usable and explainable by the team.

Business Conditions

While designing your solution, consider the following minimum business conditions:

1. A borrowing request is associated with a library member, a book title, a borrowing start date and an expected return date.
2. Each book title may have one or more physical copies. Availability must be determined using copies that are not already reserved, issued or otherwise unavailable for the requested period.
3. A physical copy must not be committed to overlapping active borrowing requests, confirmed reservations or active loans.
4. A borrowing request must be prevented from being submitted if no copy of the selected book is available for the requested period.
5. A copy of a book may be unavailable because of damage, repair, loss, reference-only use or another operational restriction.
6. A successfully submitted borrowing request must pass through a review process before it becomes a confirmed reservation or loan.
7. A rejected borrowing request is considered closed. Rejecting the request must release any associated book-copy commitment so that the copy becomes available for another valid request. To submit revised borrowing information, the requester must create a new request.
8. An appropriate end user may cancel a confirmed reservation before the book is issued. Cancelling the reservation must release the associated book copy so that it becomes available for another valid request.
9. Returning an issued book must release the associated copy for another valid request, unless the copy is marked unavailable during the return process.
10. Partial fulfilment and automatic alternative-book recommendations are not required. If an available copy of the selected book cannot satisfy the request, the request must not proceed, and the system should communicate the reason clearly.

Your team may introduce additional assumptions, provided they are documented and applied consistently.

Initial Library Data

Use the following as initial business data. Teams may add reasonable details or additional sample records for testing and demonstration.

Book Title	Category	Total Copies	Current Information
Clean Code	Software Engineering	4	2 copies issued until 4 Aug 2026
Introduction to Algorithms	Algorithms	3	1 copy reserved from 4 Aug 2026 to 8 Aug 2026
Design Patterns	Software Design	2	1 copy unavailable due to damage
Database System Concepts	Databases	5	All copies available
Artificial Intelligence: A Modern Approach	Artificial Intelligence	3	1 copy issued until 4 Aug 2026 and 1 copy reserved from 6 Aug 2026 to 7 Aug 2026

Persistence can be in-memory, local files, JSON, CSV, SQLite or another lightweight local approach.

Constraints

- 90 minutes to complete.
- Build a locally runnable prototype with a UI (Streamlit, web-based, or any framework).
- Use Python as the preferred language. C++ or Java are also acceptable.
- No cloud deployment, external APIs, paid services, complex infrastructure, or access to university production systems.
- Authentication is optional. If implemented, explain why it was prioritized.
- Each team must use a shared GitHub repository reflecting meaningful and incremental collaboration. Commit count alone will not measure individual contribution; evaluators may use repository history as supporting evidence of how the team divided, integrated and validated its work.

Deliverables

1. **Working Prototype**: A locally runnable application demonstrating a meaningful library lending lifecycle with connected workflows, consistent handling of borrowing requests, reservations, loans and book-copy states, relevant business validations, and conflict or failure handling. At least one working end-to-end flow must be demonstrated. A smaller, reliable and well-explained solution may score higher than a larger application containing generated code that the team cannot validate or explain.
2. **GitHub Repository**: Must contain source code, run instructions (prerequisites, startup, setup), documented assumptions, scope prioritized, features excluded, important trade-offs, known limitations, and the consulting pitch.
3. **Two-Slide Consulting Pitch:**
    a. Slide 1 — Problem Understanding: Your understanding of the problem, stakeholders/personas identified, workflows prioritized, important business rules discovered, key assumptions, and scope selected for 90 minutes.
    b. Slide 2 — Solution & Engineering Decisions: High-level approach, important design decisions, key trade-offs, testing/validation approach, known limitations, and what the team would improve next.
4. **Final Demonstration Covering**: business problem understanding, stakeholders and workflows identified, the working application, a successful end-to-end scenario, a conflict/invalid/failure scenario, design decisions, testing performed, assumptions and trade-offs, known limitations, and recommended next steps. Evaluators may provide an additional scenario and may direct questions to any team member — each member should be prepared to explain their contribution, one business rule, one part of the solution, one assumption or trade-off, and how their work was validated.

Use of AI Tools

This is an open-book, AI-assisted software engineering challenge. Teams may use GitHub Copilot, Copilot Chat, Claude, GPT-based models, IDE assistance, official documentation, internet search, and AI-generated code, tests, diagrams, documentation and presentation content.

Using AI is not considered cheating. However, the team remains responsible for understanding the problem, identifying requirements, choosing the solution design, reviewing generated output, detecting incorrect or unnecessary suggestions, running and validating generated code, ensuring the implementation matches the business problem, and explaining every submitted artefact.

The objective is not to demonstrate how quickly AI can generate an application. The objective is to demonstrate how effectively the team can direct, question, validate and improve AI-assisted work while retaining ownership of the final solution.

Evaluation Criteria

Criterion	Weight	What We Look For
Problem Understanding	20%	Did the team identify stakeholders, workflows, business rules and scope from the problem statement?
Implementation & Completeness	20%	Does the solution work? How many flows are functional? Is the code well-structured and explainable?
AI Usage	20%	How effectively did the team leverage AI for design, code, iteration and validation?
Collaboration & Communication	20%	How well did the team collaborate? Is the consulting pitch clear and reasoned?
Quiz	20%	Hackathon quiz conducted earlier

Bonus Feature

If your core system is functional and the primary use cases are working, implement one additional capability that enhances the system. Think about what would make this tool genuinely more useful for a campus community — notifications, analytics, calendar integration, smart suggestions, or something else entirely. Creativity counts.

> Note: Bonus is only considered if the core functionality is demonstrably working.

Duration: 90 minutes
Team Size: As announced
Tools: VS Code + GitHub Copilot